FILESENSE

Native desktop design system + product requirements

A real desktop utility for Windows, macOS, and Linux — not a web dashboard in a window.

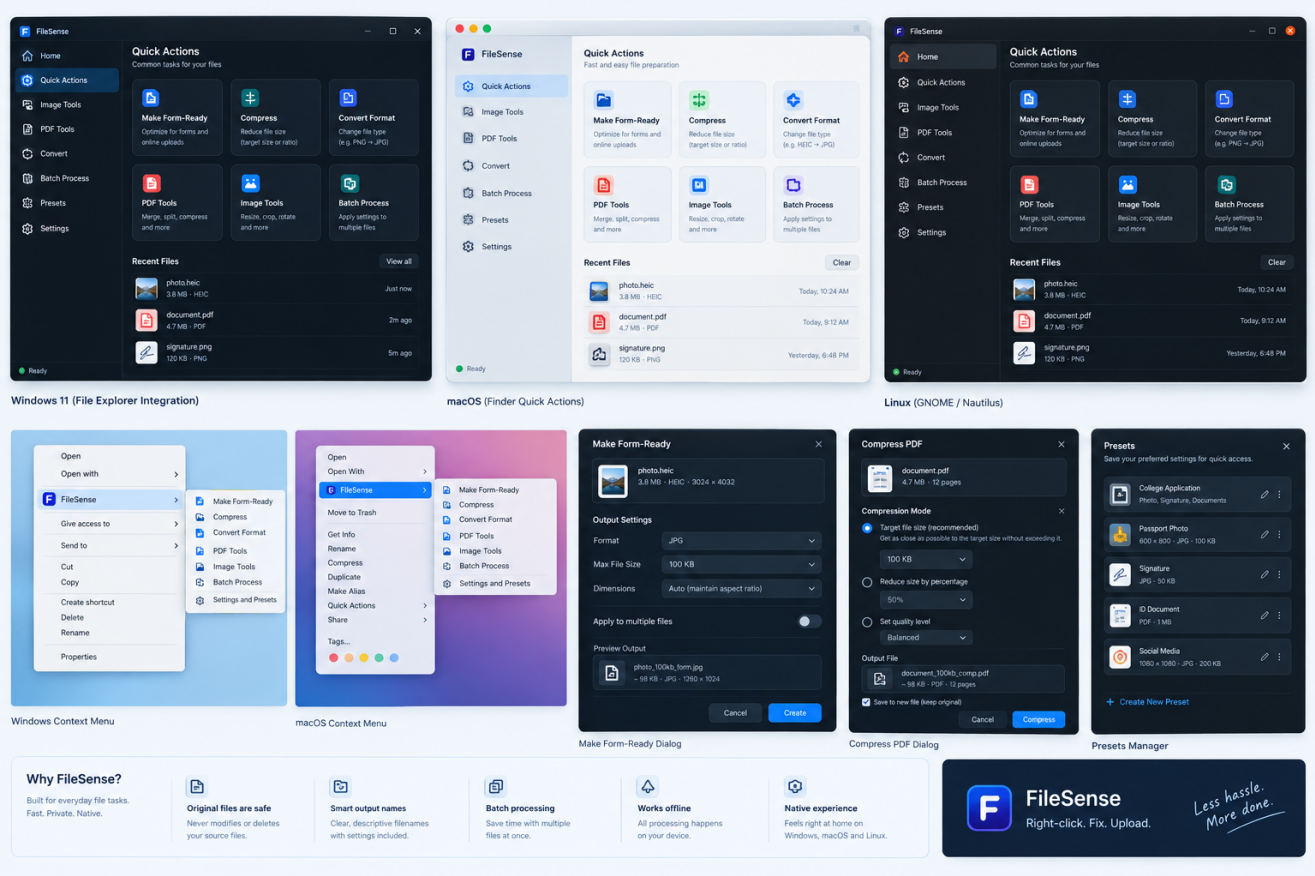
Design mandate
Every control must have a reason to exist. Each OS should feel native. Every core workflow must be tested. Originals are always preserved.

DESIGN / UX / ENGINEERING HANDOFF • v2.0 • 20 OCT 2026

This version corrects the design direction: no generic web-style card dashboard as the primary interaction model. The attached UI image is a concept reference only; implementation must follow the platform-native specifications and validation gates in this document.

1. Visual concept — use as direction, not literal UI

The concept board illustrates a shared product identity across three operating systems, plus the focused dialogs and context-menu entry points. The final application must use actual native controls and platform conventions rather than drawing imitations of them.



Important: this is a static concept image. It is not proof that the UI is native, accessible, correctly sized, or tested. Claude must treat it as a mood board and build real UI prototypes that can be run and evaluated.

2. Product and design north star

FileSense helps people convert, compress, resize, and prepare files for real upload requirements directly from the file manager. The primary experience is contextual and task-oriented: right-click → choose a meaningful action → review the output → create a copy.

Design principles

- **Native before branded:** use OS-standard title bars, controls, menus, focus rings, keyboard shortcuts, file pickers, accessibility APIs, and theme behavior. FileSense branding should not fight the host OS.
- **One task per dialog:** compression and form preparation should be focused workflows, not mini dashboards full of unrelated tiles.
- **Progressive disclosure:** show the few controls needed for the selected task. Place advanced controls behind a clearly labelled disclosure.
- **Outcome clarity:** explain target size, estimated output, actual result, and any trade-off in plain language.
- **Safe by default:** original files are never modified, deleted, or overwritten. This must be enforced by the processing layer, not only by UI copy.
- **Quiet visual hierarchy:** use typography, spacing, alignment, and native selection states before adding decoration, gradients, badges, or cards.
- **Accessible by construction:** keyboard-only operation, screen-reader labels, scalable text, visible focus, reduced-motion respect, and sufficient contrast.
- **Platform truth:** each platform may have different entry points and layout. Consistency means consistent behavior, not identical-looking windows.

What to avoid

- A web-app layout transplanted into a desktop window: oversized cards, fake browser-like panels, unnecessary persistent sidebars, or a dashboard that users must open for every small task.
- Custom-drawn imitations of native controls when a native control is available.
- A context menu with every feature and setting exposed at once.
- Decorative charts, icons, gradients, or status pills that do not help a user make a decision.
- Controls with unclear effects, redundant settings, or values that are not explained.

3. Platform-specific design requirements

Platform	Visual / interaction requirements	Integration requirements	Prototype validation
Windows 11	Follow Fluent/WinUI conventions; native title bar and menus; Windows keyboard conventions; light/dark/system theme; high-contrast mode; familiar file dialogs.	Explorer context-menu entry; handle modern menu behavior; keep shell handlers fast and delegate work to app/helper.	Test keyboard navigation, 100–200% scaling, long paths, Unicode paths, multi-select, install/uninstall cleanup, standard-user permissions.
macOS	Use system typography and standard macOS controls; respect traffic-light/title-bar conventions; native sheets/dialogs where appropriate; menu-bar and keyboard conventions.	Prefer Finder Quick Actions / Services or Shortcuts-compatible integration; app also supports Open With and drag/drop.	Test Apple Silicon and Intel if supported, VoiceOver, Dark Mode, permission prompts, notarized distribution, Finder multi-selection.
Linux	Respect desktop toolkit/theme where feasible; do not pretend GNOME and KDE are identical; use standard dialogs and portal APIs where appropriate.	Start with GNOME/Nautilus and KDE/Dolphin; document other file managers as best-effort; provide CLI/open-with fallback.	Test GNOME and KDE separately, common packaging, Wayland, file permissions, theme behavior, service-menu install/uninstall.

Architecture implication

Use a shared processing engine and shared task/validation model, but separate platform integration adapters. Avoid building one fake “universal native UI” that ignores each OS. A cross-platform framework may host the app, but critical controls and integration points should use platform-native components where practical.

Support matrix must be explicit

- Declare exact supported OS versions and distributions. Do not say “supports Linux” without listing tested environments.
- If a file-manager integration is unavailable, provide an obvious fallback: Open With, drag-and-drop, or the main app.
- Never block the file manager while compression is running.
- Installers must provide a clear enable/disable option for context-menu integration and uninstall cleanly.

4. Core workflow specifications

A. Make Form-Ready — primary differentiator

- Entry point: the first FileSense item in the context menu, labelled “Make Form-Ready…”.
- Step 1: show selected files and current properties only where relevant (format, dimensions, size, pages).
- Step 2: choose a named preset or Custom. Presets must state their constraints; they do not guarantee compliance with every website.
- Step 3: show format, maximum file size, dimensions/crop policy, and PDF readability preference. Keep advanced settings collapsed by default.
- Step 4: show the exact output filename and destination before execution.
- Step 5: show actual output size/dimensions, whether the target was met, and actions to open the result or containing folder.

B. Image compression

Control	Why it exists	Behavior / validation
Mode: Target size / Reduce by % / Quality	These express different user goals.	Mutually exclusive modes; changing mode changes visible controls and helper text.
Target size + unit	Meet form upload limits.	Maximum, not promise. Use iterative compression; resizing only when allowed.
Output format	Some formats are better suited to photos, transparency, or compatibility.	Recommend an option but explain relevant trade-offs; warn about transparency/animation loss.
Resize if needed	Smaller dimensions can help reach a target.	Off or constrained by default unless preset permits it; show resulting dimensions.
Quality floor	Prevent unreadable or unacceptable output.	Use understandable labels such as Balanced / Higher quality / Smaller file, backed by tested parameters.
Output preview	Prevent confusing names and surprise destinations.	Preview actual collision-safe name; do not claim a target-size suffix until it is achieved.

C. PDF compression

- Offer Target size, Reduce by %, and readability preference as distinct settings.
- Prefer preserving text and vector elements; downsample embedded images only when appropriate.
- Expose a simple “Balanced / Smaller file / Preserve readability” choice. Put advanced image DPI and optimization options behind Advanced.
- For scanned PDFs, warn when aggressive compression could make small text or signatures difficult to read.
- Report page count and actual resulting size. If the target cannot be reached safely, say so and offer alternatives.

5. Context menus, naming, and non-destructive behavior

Keep the OS context menu short. It is a launch point, not the entire application.

Menu entry	Show when	Reason
Make Form-Ready…	Supported image/PDF selection	Highest-value guided action; handles multiple constraints together.
Compress…	Images/PDFs	Common direct task with one focused dialog.
Convert Format…	Convertible files	Avoids listing every format as a top-level menu item.
Image Tools ■	Image selected	Group resize, crop, rotate, flip.
PDF Tools ■	PDF selected	Group merge/split/rotate/extract where valid.
Batch Process…	Multiple compatible files	Explicit multi-file flow with per-file status.
Presets / Settings	Optional submenu or app window	Keep preferences away from everyday actions.

Filename rules

Operation	Example	Rule
PNG → JPG	photo_png_to_jpg.jpg	Describe actual conversion.
HEIC → JPG	photo_heic_to_jpg.jpg	Keep the new extension correct.
Target size achieved	image_200kb_comp.jpg	Use target-size suffix only if actual output is at or below target.
Target not achieved	image_comp.jpg	Do not mislead with _200kb when it is larger.
Resize	signature_600x600.jpg	Dimensions must match actual output.
Compound operation	photo_heic_to_jpg_200kb_comp.jpg	Use stable suffix order.

Collision handling: if the output exists, append _2, _3, etc. Never overwrite by default. Default output folder is beside the original; allow a custom folder. Write to a temporary file, validate it, then move it into place using a collision-safe operation.

Hard acceptance rule: automated tests must verify that source files remain byte-for-byte unchanged after success, failure, cancellation, and app crash/recovery scenarios.

6. Component audit — every element needs a job

Component	Keep it if…	Remove / rethink it if…
Title bar / window chrome	It follows the OS and gives the user standard window controls.	It is a custom imitation with broken accessibility or platform behavior.
File summary row	It helps confirm the right input and provides relevant metadata.	It repeats the filename in several places or adds irrelevant details.
Mode selector	It helps the user choose a distinct goal.	Modes overlap or a single mode would be clearer.
Numeric field + unit selector	Exact target sizes matter; keyboard entry is supported.	A slider is the only way to set an exact value.
Quality/readability choice	It exposes a meaningful trade-off backed by tested behavior.	Labels are vague or options produce indistinguishable outcomes.
Advanced disclosure	Rare settings exist but should not overwhelm most users.	It hides essential controls or becomes a dumping ground.
Output filename preview	It prevents surprises and shows collision-safe naming.	It is not synchronized with actual output behavior.
Progress indicator	Operations take long enough to warrant feedback.	It gives false precision or blocks cancellation.
Result summary	It confirms actual size, dimensions, target status, and destination.	It shows only a generic “Success” toast.
Cards / tiles	They are useful for a small number of independent destinations on a home screen.	They turn a focused dialog into a marketing dashboard.
Icons / badges	They improve recognition or convey meaningful state.	They are decorative or duplicate nearby text.

Design-review question for every component: “What decision does this help the user make, what action does it enable, and what happens if we remove it?” If the team cannot answer clearly, remove or redesign it.

7. Design quality and testing plan

Do not call the UI polished because a screenshot looks good. It is polished only after the real application is exercised at realistic sizes, with keyboard and accessibility checks, and across the supported platforms.

Required design review gates

Gate	Test cases	Pass condition
Visual layout	Minimum supported window size; long filenames; long translations if localized; small and large text; light/dark themes.	No clipped labels, overlapping controls, unexplained blank areas, or horizontal scrolling for core tasks.
Keyboard	Tab/Shift+Tab, Enter, Escape, arrow keys in menus, platform-standard shortcuts.	All core tasks can be completed without a mouse; focus is always visible and logical.
Accessibility	Windows Narrator/NVDA as appropriate; macOS VoiceOver; Linux screen reader where supported; contrast checks.	Inputs have meaningful accessible names; status/progress is announced; state is not colour-only.
File safety	Existing destination, read-only source, no disk space, cancellation, malformed input, app interruption.	Original unchanged; no silent overwrite; useful error and recoverable state.
Compression quality	Photos, screenshots, transparent images, HEIC, scanned PDFs, text PDFs, already-compressed files.	Output is visually reviewed; target status accurate; quality-floor behavior predictable.
Platform integration	Install, disable integration, uninstall, update, multiple selection, unsupported file type.	No stale menu items; no shell hangs; fallback workflow documented.
Batch processing	Mixed formats, one corrupt file among valid files, cancel mid-batch.	Each item reports status separately; successful outputs remain valid; originals untouched.

Testing artifacts Claude should produce

- Runnable prototype for each supported OS, not just a screenshot.
- Screenshot set at normal and scaled UI sizes for each major workflow.
- Platform-specific checklist with tested OS version, toolkit, and integration path.
- Automated unit/integration tests for naming, collision handling, target attainment, and source-file preservation.
- Manual quality matrix with representative files and before/after comparisons.
- A list of known limitations and unsupported combinations visible to users.

8. Architecture, scope, and phased plan

Recommended direction: shared processing engine with native integration adapters. Tauri + Rust is a candidate, not a foregone conclusion; compare it against alternatives in a short proof of concept before committing.

Layer	Responsibility	Decision to validate
Platform adapter	Context menu / Quick Action / service-menu entry, file paths, OS dialogs, app launch.	Real integration feasibility and installer behavior per OS.
Shared task model	Operation parameters, validation, queue, progress, cancellation, results.	Stable API shared by UI and CLI/fallback.
Image backend	Decode/encode, resize, metadata/color profiles, format support.	HEIC licensing/support, animation, alpha, ICC profiles.
PDF backend	Compression, page operations, preservation of text/vector data.	License compatibility, security, readability, cross-platform packaging.
File-safety layer	Temp files, output naming, collision prevention, atomic finalize, cleanup.	Enforce source immutability independently of UI.
Packaging/update	Install, sign/notarize, optional integration, update, uninstall.	Per-OS distribution and permission model.

Phased roadmap

Phase	Scope	Exit criteria
0 — Risk spike	Prototype one context action per OS; test image/PDF backend and licensing.	Known-good integration proof; dependency/licensing decision; test file corpus.
1 — MVP	Make Form-Ready; image convert/compress/resize; PDF compress; filename preview; collision safety; presets; basic batch.	All acceptance criteria pass on each supported OS.
2 — Productivity	PDF merge/split/rotate/extract; image crop/rotate/flip; batch queue; history.	Stable multi-file behavior and recovery.
3 — Document intelligence	OCR, searchable PDFs, auto-rotate, blank-page detection, text extraction.	Language limitations and accuracy tested.
4 — Advanced	Office conversions, watermark/page numbers, repair, advanced tools.	Per-feature quality, security, and license review.

9. Handoff prompt for Claude

```
You are the product designer and senior cross-platform desktop engineer for FileSense. Use this PDF as the authoritative design and product brief. The previous design direction was too web-like. Redesign it as a genuinely native-feeling desktop utility. Do not simply copy the concept image. Use platform conventions and real controls: - Windows: Fluent/WinUI conventions and Explorer integration. - macOS: native macOS controls, Finder Quick Actions, standard dialogs, system typography and keyboard behavior. - Linux: explicitly supported environments first (GNOME/Nautilus and KDE/Dolphin), with native dialogs/theme behavior and documented fallback. For every screen and component, document: 1. The user need it serves. 2. Why the component is present. 3. Default state, advanced state, validation, errors, loading, success, and cancellation. 4. Keyboard and accessibility behavior. 5. Platform-specific differences. 6. Tests required before calling it complete. Hard requirements: - Never modify, delete, or overwrite source files. - Always create a separate output file. - Default output folder is beside the original; allow a custom destination. - Resolve filename collisions with _2, _3, etc. - Preview the exact output filename. Do not claim a target size was achieved unless actual output meets it. - Target size, percentage reduction, and quality level are distinct modes. - If a target cannot be met above the quality/readability floor, explain it and offer safe alternatives. - Core processing is local by default. - Keep context menus short and task-focused. - Do not claim universal Linux file-manager support. Deliverables, in order: A. Identify ambiguities and risky assumptions. B. Technical spike plan and dependency/license review. C. Platform-specific interaction maps. D. A component inventory with rationale; remove anything decorative or redundant. E. Runnable prototypes for the supported OSes, or clearly state which prototypes cannot be run in the current environment. F. Acceptance tests, accessibility checklist, file-safety tests, and visual QA plan. G. MVP backlog with measurable exit criteria. Do not implement the full roadmap before the technical spike passes. A polished screenshot is not evidence of a polished, tested application.
```

End of FileSense Native UI Design + PRD v2.0